EXPT NO: 1	FUNDAMENTALS OF PYTHON AND OPENCV
DATE: 02.12.25	

A.INTRODUCTION TO PYTHON FOR COMPUTER VISION:

COMPUTER VISION:

Computer Vision (CV) is a branch of Artificial Intelligence (AI) that helps computers to interpret and understand visual information much like humans. Tasks such as object detection, image classification, face recognition, and motion analysis rely heavily on CV techniques.

1.Why Python is widely used in CV?

Python is a widely adopted language for computer vision due to a combination of factors:

1. Rich Ecosystem of Libraries and Frameworks:

OpenCV: The Open Source Computer Vision Library (OpenCV) provides a comprehensive set of tools for image processing, feature detection, object recognition, and more. Python bindings for OpenCV make it easily accessible.

Numerical Libraries: Libraries like NumPy and SciPy facilitate efficient handling and manipulation of image data, which is often represented as multi-dimensional arrays.

Deep Learning Frameworks: Python offers first-class APIs for popular deep learning frameworks such as TensorFlow, PyTorch, and Keras, which are essential for developing and training advanced computer vision models like Convolutional Neural Networks (CNNs).

Image Processing Libraries: Scikit-image provides a collection of algorithms for image processing tasks, complementing OpenCV.

2. Ease of Use and Readability:

Python's clear, concise syntax and high readability make it relatively easy to learn and use, particularly for those new to computer vision. This accelerates development and debugging.

3. Strong Community Support:

Python boasts a large and active community of developers, leading to extensive documentation, tutorials, online forums, and readily available code examples for various computer vision tasks.

2.Basic syntax and data types

1. Print Statement

Used to display output or debug image processing steps.

```
print("Computer Vision Program Started")
```

2. Indentation

Python uses **indentation instead of braces** {} to define code blocks.

```
if x > 100:
```

```
    print("High pixel value")
```

3. Import Statements

Every CV program starts by importing libraries.

```
import cv2
import numpy as np
```

4. Variables

Variables store image paths, pixel values, thresholds, etc.

```
image_path = "photo.jpg"
threshold = 120
```

5. Comments

Used to explain CV steps.

```
# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

6. Functions

Functions group image processing operations.

```
def show_image(img):
    cv2.imshow("Output", img)
```

7. Conditional Statements

Used for thresholding and decision making.

```
if pixel > 128:
    pixel = 255
else:
    pixel = 0
```

8. Loops

Used for iterating through frames in video or datasets.

```
for i in range(10):
    print("Frame:", i)
```

9. Reading Input

Python syntax for reading an image:

```
img = cv2.imread("image.jpg")
```

10. Displaying Output

Syntax to show the image:

```
cv2.imshow("Result", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

DATA TYPES:

1. Integer (int)

Represents whole numbers (no decimals). Used for pixel values (0–255), image dimensions, thresholds.

```
pixel_value = 255
height = 480
```

2. Floating-Point (float)

Represents numbers with decimals. Used for normalization, scaling, ratios, filter values.

```
scale = 0.5
learning_rate = 0.001
```

3. String (str)

Represents text.Used for image file paths or labels.

```
image_path = "image.jpg"
label = "cat"
```

4. Boolean (bool)

Represents True or False. Used for checking conditions in CV programs.

```
is_blurry = True
```

5. List (list)

Ordered collection of values.Used for storing pixel values, coordinates, or multiple images.

```
colors = [255, 0, 0]
```

6. Tuple (tuple)

Similar to lists but **immutable** (cannot be changed).Used for fixed values like image shape or color channels.

```
size = (640, 480)
color = (0, 255, 0)    # BGR green
```

7. Dictionary (dict)

Stores data as key–value pairs.Useful for storing metadata or configuration settings.

info = {"width": 640, "height": 480}

8. NumPy Array (numpy.ndarray)

Most important data type in Computer Vision.Images in OpenCV are stored as NumPy arrays (matrices).

- **Grayscale image** → 2D array
- **Color image (BGR/RGB)** → 3D array (height × width × channels)

import numpy as np

img_matrix = np.array([[0, 255], [100, 200]])

3. Important libraries

1.**NumPy** — handles arrays, matrices (backbone of OpenCV)

2.**OpenCV (cv2)** — image processing, object detection, filtering

3.**Matplotlib** — visualization of images and graphs

4.**Pillow** — basic image manipulation

5.**TensorFlow / PyTorch** — deep learning models for CV

6.**Scikit-image** — scientific image analysis

B. OPENCV FUNDAMENTALS

1.Overview of OpenCV and its applications

OpenCV (Open Source Computer Vision Library) is one of the most powerful and widely used libraries for:

- Image processing
- Face detection and recognition
- Video analysis
- Object detection
- Feature extraction
- Machine learning tasks
- Image transformations

OpenCV supports more than **2500+ optimized algorithms**.

2.Installation steps (pip, environment setup)

```
[2]
✓ 10s
#pip
!pip install opencv-python
!pip install numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
[3]
✓ 4s
#environment setup
!pip install numpy matplotlib opencv-python
import cv2
print("OpenCV Version:", cv2.__version__)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
OpenCV Version: 4.12.0
```

3.Reading, writing, displaying images

```
import cv2
from google.colab.patches import cv2_imshow
# 1. Read an Image
img = cv2.imread("image.jpg")
if img is None:
    print("Error: Image not found!")
else:
    print("Image Loaded Successfully")
# 2. Display Image (Colab uses cv2_imshow)
cv2_imshow(img)
# 3. Save Image
cv2.imwrite("saved_output.jpg", img)
print("Image Saved as saved_output.jpg")
```

```
Image Loaded Successfully
```

```
Image Saved as saved_output.jpg
```

4.Color models (BGR, RGB, Grayscale, HSV, LAB)

```
#color models
# BGR to RGB
rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2_imshow(rgb_img)
# BGR to GRAYSCALE
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2_imshow(gray_img)
# BGR to HSV
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
cv2_imshow(hsv_img)
# BGR to LAB
lab_img = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
cv2_imshow(lab_img)
```

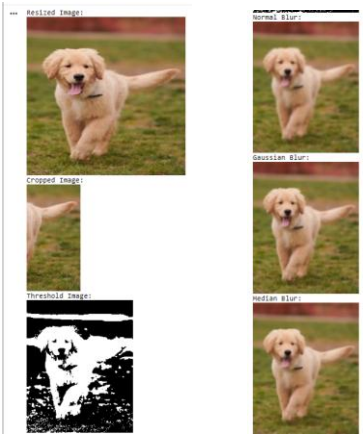
5.Basic image operations: resizing, cropping, thresholding, filtering

```
# Resizing
resized = cv2.resize(img, (300, 300))  # Resize to 300x300
print("Resized Image:")
cv2_imshow(resized)
#  Cropping
# Crop region: [y1:y2, x1:x2]
cropped = img[50:300, 100:350]
print("Cropped Image:")
cv2_imshow(cropped)
# Thresholding (Binary)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY)
print("Threshold Image:")
cv2_imshow(thresh)
# Filtering / Blurring
# (a) Normal Blur
blur = cv2.blur(img, (5, 5))
print("Normal Blur:")
cv2_imshow(blur)
# (b) Gaussian Blur
gaussian = cv2.GaussianBlur(img, (5, 5), 0)
print("Gaussian Blur:")
cv2_imshow(gaussian)
# (c) Median Blur
median = cv2.medianBlur(img, 5)
print("Median Blur:")
cv2_imshow(median)
```

How can I install Pyt

What can I help

+



6.Understanding OpenCV functions and image matrices

```
[9]
✓ 0s
#  Image Matrix Properties
print("Image Shape:", img.shape)     # (height, width, channels)
print("Image Size (total pixels):", img.size)
print("Image Data Type:", img.dtype) # Usually uint8
# Access Pixel Values
pixel = img[100, 150]
print("Pixel value at (100,150):", pixel)  # BGR format
# Modify pixel (example: set to blue)
img[100, 150] = [255, 0, 0]
print("Pixel value at (100,150) after modification:", img[100, 150])
# 4. Access a Region of Interest (ROI)
roi = img[50:150, 100:250]  # [y1:y2, x1:x2]
print("ROI shape:", roi.shape)
cv2_imshow(roi)
# Understanding OpenCV Functions
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print("Grayscale Image Shape:", gray.shape)
cv2_imshow(gray)
# Example: Flip image horizontally
flip_h = cv2.flip(img, 1)  # 1 = horizontal
cv2_imshow(flip_h)
```

```
Image Shape: (251, 201, 3)
Image Size (total pixels): 151353
Image Data Type: uint8
Pixel value at (100,150): [171 193 228]
Pixel value at (100,150) after modification: [255   0   0]
ROI shape: (100, 101, 3)
```

```
Grayscale Image Shape: (251, 201)
```

C. System Requirements & Software Setup,

1.Python version

```
import sys
print("Python version:", sys.version)

Python version: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
```

2.OpenCV installation commands

```
[11]
✓ 4s
!pip install opencv-python numpy matplotlib

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

3.IDE or Notebook environment (VS Code / PyCharm / Jupyter Notebook)

VS CODE- light weight , supports virtual environments and extensions

PYCHARM- full featured IDE for large projects

JUPYTER NOTEBOOK- interactive , displaying image inline

GOOGLE COLAB- cloud based , no local setup, GPU support

4.Hardware requirements (processor, RAM, etc.)

PROCESSOR -dual core

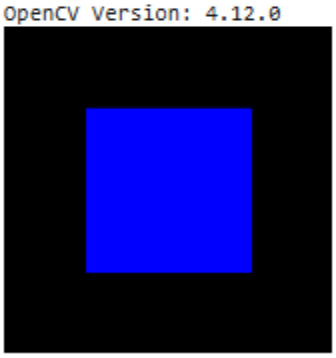
RAM – 4GB

STORAGE – 200MB

5.How to test installation with a simple program

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
print("OpenCV Version:", cv2.__version__)
img = np.zeros((200, 200, 3), dtype=np.uint8)
cv2.rectangle(img, (50, 50), (150, 150), (255, 0, 0), -1)
cv2_imshow(img)
print("Installation test completed successfully!")
```

```
OpenCV Version: 4.12.0
```



```
Installation test completed successfully!
```